

SliceMatic

Stage 1 Submission

Product Requirements Document
& Business Unit Economics

*FDE Academy Programme - PizzaFlow Applied Project
Digital Ordering System for SliceMatic, New Ashok Nagar, Delhi*

Prepared by - Group 3

Mouli Murakambattu · Shaik Mohammed Pasha · Aman Sharma · Saurabh Sekhar · Sushant Kumar

SliceMatic Ordering System — Product Requirements Document

Project: SliceMatic Digital Ordering System

Stage: 1 (Part A of 2) — Product Requirements Document

Prepared for: SliceMatic (New Ashok Nagar, Delhi) — Founder: Mr. Rajan Sharma

Document status: Draft v1.0 for client review

Date: 23 June 2026

1. Product Vision

SliceMatic's ordering system replaces error-prone manual phone ordering with a guided digital flow that validates customer details, builds an itemised bill (pricing, quantity discounts, 18% GST and payment), and persists every order in a structured, analysable format. It solves two problems at once:

1. The **customer**, who gets a fast, consistent, transparent ordering experience
2. The **outlet owner**, who gets operational consistency (no manual pricing or GST errors, no staff tied to a phone line) plus the order-level data needed to run the unit economics of the business.

2. Functional Requirements

Each requirement has an ID, the requirement, and its acceptance criteria.

ID	Requirement	Acceptance criteria
FR-1.1	Capture customer name	Alphabets and spaces only; 2–40 characters; reject anything else with a specific message and re-prompt
FR-1.2	Capture phone number	Exactly 10 digits; first digit 6, 7, 8 or 9; reject otherwise and re-prompt
FR-1.3	Log session timestamp	Record the timestamp at the start of each new ordering session
FR-2.1	Load menu from files at startup	Load Types_of_Base.txt, Types_of_Pizza.txt, Types_of_Toppings.txt; each line formatted ID;Name;Price
FR-2.2	Display menu	Show each category as a numbered list with name and price in INR
FR-2.3	No hardcoded menu	Menu comes only from the files; editing or replacing the files shows the new menu with no code change
FR-3.1	Item selection	Customer selects one Base, one Pizza and one Topping (topping count pending — see Q3); each must resolve to a valid menu item
FR-3.2	Selection by number	Selection by item number only; reject out-of-range numbers, letters and empty input with a specific message; re-prompt

ID	Requirement	Acceptance criteria
FR-4.1	Quantity	Accept an integer from 1 to 10; reject 0, negatives, values above 10, floats and non-numeric input; re-prompt
FR-4.2	Capacity cap	Maximum 10 pizzas per order; reject any value above 10 with an explanation
FR-5	Pricing engine	Compute the bill per the pricing table below
FR-6.1	Itemised bill	Show Base + Pizza + Topping per unit, quantity, unit price and subtotal
FR-6.2	Bill totals	Show discount (if applicable), GST @ 18% on the post-discount total, and final payable
FR-6.3	Bill rendering	Render as a structured table component (not plain text); aligned columns, totals clearly marked
FR-7.1	Payment modes	Offer exactly three modes: Cash, Card, UPI
FR-7.2	Payment confirmation	Confirm the chosen mode; show a mode-appropriate confirmation message
FR-7.3	Payment validation	Reject any invalid payment selection and prompt to retry
FR-8.1	Persist order	Append every completed order to orders_log.txt
FR-8.2	Record fields	timestamp, name, phone, item selections, unit prices, quantity, subtotal, discount, GST, final total, payment mode
FR-8.3	Log format	One order per block; fields pipe-separated within a line; a blank line between orders (machine-parseable)

FR-5 — Pricing Engine (computed in this exact order)

Step	Calculation
FR-5.1 Unit price	Base price + Pizza price + Topping price
FR-5.2 Subtotal	Unit price × Quantity
FR-5.3 Discount	10% of Subtotal if Quantity ≥ 5, else 0 (shown as a bill line when applied)
FR-5.4 Taxable amount	Subtotal – Discount
FR-5.5 GST	18% of the Taxable amount (charged on the post-discount total)
FR-5.6 Final payable	Taxable amount + GST
FR-5.7 Rounding	All money values to 2 decimals; INR shown on every line

*Worked example (from the client's reference model): Base Cheese Burst ₹229 + Pizza BBQ Chicken ₹379 + Topping Extra Cheese ₹69 = ₹677 unit. Qty 5 → Subtotal ₹3,385 → Discount (10%) ₹338.50 → Taxable ₹3,046.50 → GST (18%) ₹548.37 → **Total ₹3,594.87**.*

3. Non-Functional Requirements

3.1 Input Validation Rules

Field	Rule
Name	Alphabets and spaces only; 2–40 chars; not blank or whitespace-only
Phone	Exactly 10 digits; first digit one of 6, 7, 8, 9
Quantity	Integer 1–10 only; no floats, strings, 0, negatives, above 10
Menu selection	Valid list number only; in range; not letters or blank
Payment	One of 1, 2, 3 only

3.2 Quality & Reliability Requirements

ID	Area	Requirement
NFR-2	Error messages	Every rejection gives a specific, helpful message stating what was wrong and what is expected, then re-prompts — never a generic failure, never a crash
NFR-3	Edge cases	All edge cases in 3.3 handled with no unhandled exception
NFR-4	Orders log format	Pipe-separated, one order per block, blank line between blocks; stable field order (see below) so downstream analysis can parse it reliably
NFR-5	Graceful failure	Missing or malformed menu file → clear error and graceful exit (no stack trace); malformed lines validated defensively; no crash on any valid or invalid input
NFR-6	Menu-change robustness	Strip whitespace, tolerate blank lines, validate price is numeric, handle a varying number of items per file
NFR-7	Usability	Step-by-step flow (not one giant form); prices and totals always in INR; re-prompts keep the customer in context

Example error message (NFR-2): phone 1234567890 → "Phone must be 10 digits and start with 6, 7, 8 or 9. Please re-enter."

Orders log field order (NFR-4):

```
timestamp ; name ; phone ; base ; pizza ; topping ; unit_price ; quantity ; subtotal ; discount ; gst ; total ; payment_mode
```

(Shown with ; separators for readability; actual log uses the pipe character as specified in FR-8.3.)

3.3 Edge Cases (all handled with no unhandled exception)

#	Edge case	Required handling
1	Name with only spaces	Reject as invalid name; re-prompt
2	Phone with 10 digits starting with 1	Reject; require first digit 6/7/8/9
3	Quantity = 0 or 11	Reject; require an integer 1–10
4	Item selection = 0 or above menu length	Reject; require an in-range number
5	Price number entered instead of item number	Reject if out of range; treat as invalid selection
6	Empty input at any prompt	Reject; re-prompt with guidance
7	Non-integer at quantity (e.g. three, 2.5)	Reject; require an integer

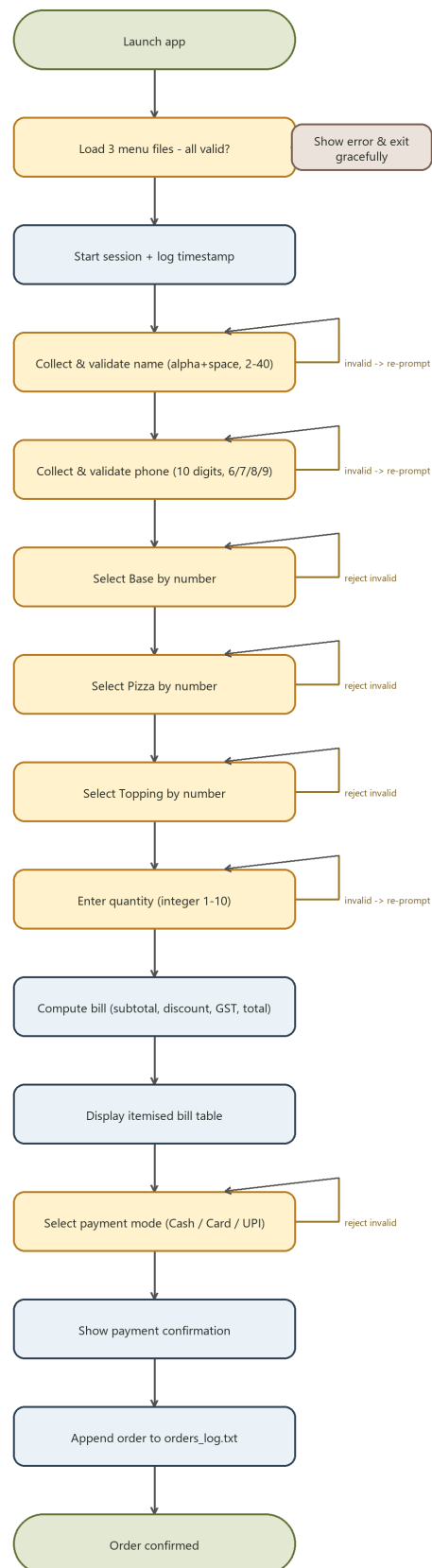
#	Edge case	Required handling
8	Menu file with a missing price field	Skip or validate the line defensively; do not crash

4. User Flow Diagrams

Two flows are documented. **4.1 — the Customer Ordering Flow** is the core journey delivered in the MVP (Stage 2). **4.2 — the Admin Flow** is a Stage 3 capability, shown here for completeness so the full system is understood end to end; it is **out of scope for the Stage 2 MVP**.

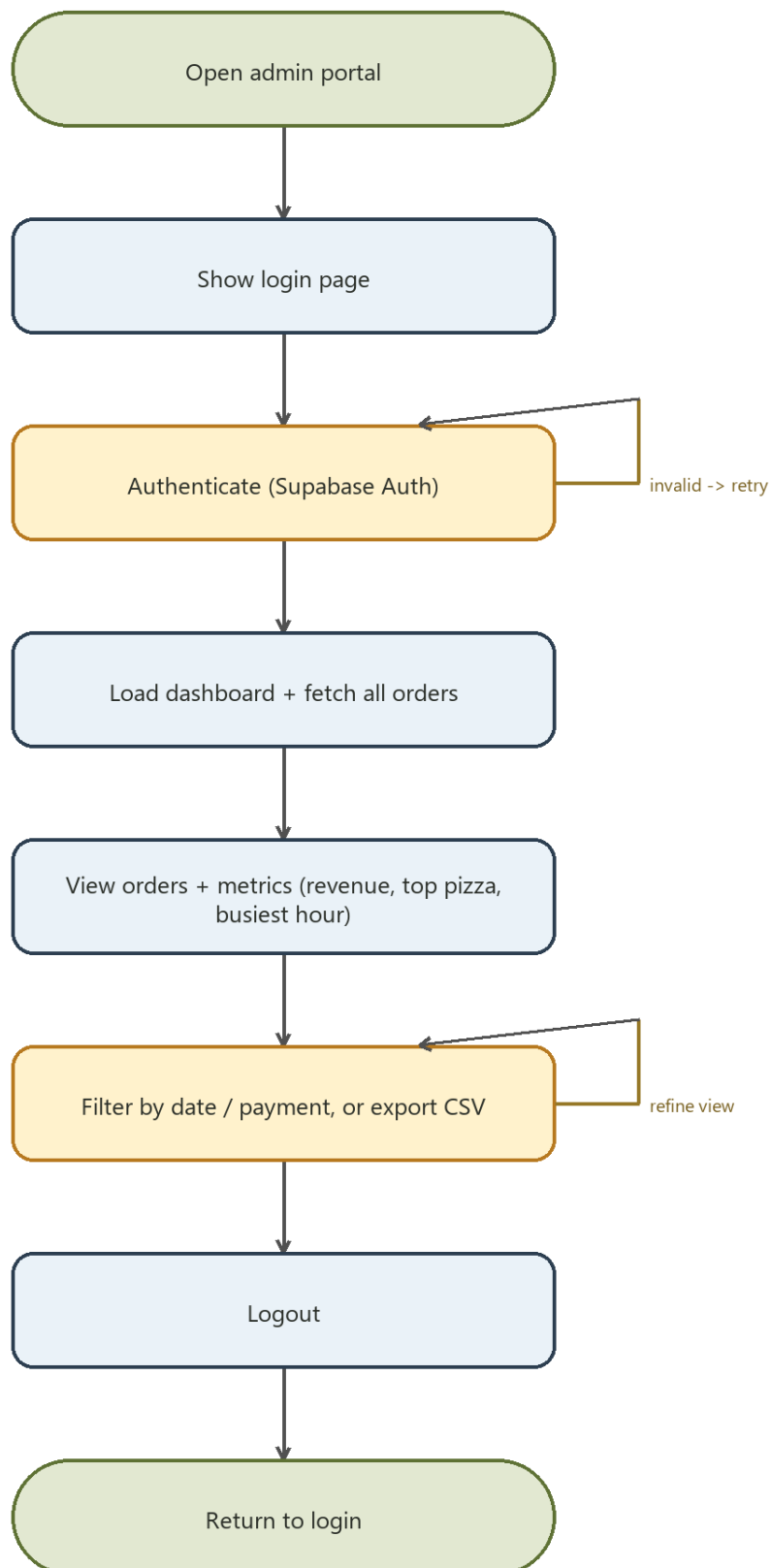
4.1 Customer Ordering Flow

End-to-end journey from launch to order confirmation, including every decision node and error branch. (Rendered as a Mermaid flowchart — displays in Notion / GitHub / most Markdown viewers.)

Figure 4.1 - Customer Ordering Flow (MVP)

4.2 Admin Flow (Stage 3 scope — not in the MVP)

The owner/admin journey: authenticated login, then a dashboard built on the persisted order data (filters, revenue and sales summaries, CSV export). Backed by Supabase Auth and the orders database in the full-stack build.

Figure 4.2 - Admin Flow (Stage 3 scope)

5. AI Feature — Conversational Ordering Interface (Stage 3 scope — Option B)

For Stage 3, SliceMatic will implement **Option B** from the brief: a conversational (chat) ordering interface powered by an LLM via **OpenRouter**, replacing the step-by-step form with natural-language ordering. All Stage 2 business logic (validation, pricing, discount, GST, persistence) still applies — the LLM handles understanding, not calculation. This section is

out of scope for the Stage 2 MVP.

5.1 Overview

The customer places an order by typing naturally — e.g. "2 cheese burst pizzas with extra cheese, paying by UPI, name Rajan, 9876543210." An LLM agent extracts the required fields, confirms each one, re-prompts for anything missing or invalid, handles vague requests, and finalises the order. If the agent cannot parse the request, it falls back to the structured Stage 2 flow.

5.2 Functional Requirements

ID	Requirement	Detail
AI-1	Natural-language intake	Customer orders in free text via a chat interface (replaces the form flow)
AI-2	Field extraction	Agent extracts the 7 required fields: name, phone, quantity, base, pizza, topping, payment mode
AI-3	Per-field confirmation	Agent reads back each extracted field for the customer to confirm or correct
AI-4	Ambiguity handling	Vague input (e.g. "something spicy") → agent suggests matching items from the live menu (Peri-Peri, Jalapenos, BBQ Chicken)
AI-5	Missing/invalid re-prompt	Agent re-prompts for any field that is missing or fails validation, one at a time
AI-6	Menu grounding	Agent may only choose bases/pizzas/toppings that exist in the live menu/DB; it cannot invent items or prices
AI-7	Deterministic pricing	Agent extracts intent only; the validated pricing engine computes subtotal, discount, GST and total — the LLM never computes money
AI-8	Business-rule parity	All Stage 2 rules enforced: name 2–40 alpha+space, phone 10-digit 6/7/8/9, qty 1–10, 10% discount at qty ≥ 5, 18% GST on post-discount total
AI-9	Confirmation before commit	Agent reads back the full order and final payable before finalising
AI-10	Persistence	Completed chat orders saved to the same orders store, tagged with order source = "chat"
AI-11	Graceful fallback	If the LLM/API is unavailable or parsing fails after a few attempts, fall back to the structured form flow

5.3 Guardrails & Non-Functional Requirements

- **No hallucinated items or prices** — the live menu is provided to the agent (system prompt or tool call); selections are validated against the DB before pricing.
- **Math stays deterministic** — pricing, discount and GST are computed by the same server-side engine as the form flow, never by the LLM.
- **Input safety** — off-topic, abusive, or prompt-injection input is deflected; the agent stays on the ordering task.
- **Graceful degradation** — LLM/API failure falls back to the Stage 2 form flow so an order can always be placed.
- **Transparency** — the final itemised bill (same format as the form flow) is shown before payment.

5.4 Model, Prompt & Documentation

- **Provider:** OpenRouter API (mandatory per brief). The specific model and rationale are documented in the README (chosen at Stage 3 — a strong instruction-following model with reliable structured extraction).
- **System prompt:** documented in the README (brief requirement). It defines the agent's role, the 7 fields to collect, the live menu, the validation rules, the instruction to confirm each field, and the rule never to compute prices.
- **Ownership:** per the brief, the first team to commit an AI feature owns it — this Option B implementation is registered as Group 3's.

***Example exchange:** Customer: "Hi, I want a cheese burst, something spicy on top, 3 of them." → Agent: "Great — Cheese Burst base. For 'something spicy' we have Jalapenos, Peri-Peri Drizzle or BBQ Chicken — which would you like? And which pizza: Pepperoni, BBQ Chicken or Paneer Tikka? Also, may I have your name and 10-digit phone number?" The agent continues until all 7 fields are valid, then confirms the itemised bill before payment.*

6. Drawbacks Analysis

An honest assessment of the system **as specified** — not just its strengths.

6.1 Architectural limitations

- **Flat-file storage does not scale.** orders_log.txt is appended sequentially. At ~1,000+ orders the file becomes slow to read, has no indexing, no querying, and no concurrency control — two simultaneous orders could interleave or corrupt a line. This is acceptable for an MVP but is the first thing that breaks at volume.
- **Single-session, single-user.** The MVP assumes one order at a time. Real delivery demand is concurrent (peak-hour bursts); the spec has no concurrency model.
- **No persistence of menu/version.** If the menu file changes between orders, past orders in the log have no record of which menu/price version was in effect.

6.2 Functional gaps for a real ordering business

- **Single-configuration orders.** The spec models one Base+Pizza+Topping × quantity. A customer who wants two different pizzas in one order cannot be served — no true cart.
- **No real payment processing.** Payment is a selection + confirmation only; no money actually moves for Card/UPI. Fine for an MVP, not for production.
- **No order tracking, cancellation, or modification** after confirmation.
- **No inventory or stock-out handling.** The system will happily sell a pizza whose ingredients are unavailable.

6.3 What breaks at 1,000 orders

- File read/parse for any analysis becomes a full-file scan.
- No deduplication or order IDs → hard to reference a specific order.
- Concurrent writes risk corrupting the log.
- No backups → a single file loss wipes all business data.

7. Cost vs Value Analysis

7.1 Estimated build effort (MVP — Stage 2 scope)

Work item	Est. effort (hrs)
Menu file loader + defensive parsing	3–4
Input validation (name, phone, qty, selection, payment)	4–5
Pricing engine (discount + GST + rounding)	2–3
Bill rendering (structured table component)	2–3
Payment flow + confirmations	1–2
Order persistence (parseable log)	2
Edge-case hardening + testing (all 8 cases)	4–5
UI assembly / step-driven flow	3–4
Total (MVP)	~21–28 hrs

(Indicative full-stack Stage 3 effort is materially larger — frontend, database, auth, dashboard, AI feature — and is scoped separately.)

***Note — Admin dashboard:** the admin flow (see 4.2) is a **Stage 3** capability and is **not** included in the ~21–28 hr MVP estimate above. Its effort (Supabase Auth, dashboard metrics, date/payment filters, CSV export) is part of the separately-scoped full-stack build.*

7.2 Measurable value to the outlet

- **Operational efficiency:** removes manual price/discount/GST math (eliminates billing errors); frees the counter/billing staff from being tied to a phone line; consistent bills every time.
- **Data captured:** every order recorded in a parseable format → enables AOV, top-selling items, peak-hour analysis, weekday/weekend split, and break-even tracking — the exact inputs the owner needs to manage unit economics (Part B). Today this data does not exist.
- **Customer experience:** faster, transparent ordering with an itemised bill; fewer disputes over price; consistent discount application.
- **Strategic:** the order log is the foundation for the Stage 3 admin dashboard and AI features (recommendations / demand forecasting), which directly target higher AOV and better capacity

planning.

Net read: ~1 working-week of build effort produces a system that removes recurring billing errors, recovers staff time, and — most importantly — starts capturing the order data the business currently has none of. High value-to-effort ratio for the MVP.

8. Expected Outcomes & Success Metrics

What success looks like — the measurable outcomes the system must deliver. These tie directly to the unit economics in Part B (the order data captured here is what powers AOV, peak-hour and break-even tracking).

#	Outcome	Success metric / target
O-1	Accurate billing every time	100% of bills correct — discount applied at qty ≥ 5 , 18% GST on post-discount total; zero manual calculation errors
O-2	No crashes on any input	0 unhandled exceptions across all 8 edge cases and the grader's swapped menu files
O-3	Every order captured	100% of completed orders written to the log in a parseable, consistent format
O-4	Menu changes need no code	A new or edited menu file loads at runtime with zero code changes
O-5	Faster, consistent ordering	Order completed in under ~2 minutes; consistent discount and GST on every order
O-6	Staff time recovered	Counter/billing staff freed from manual phone order-taking
O-7	Owner can run the numbers	AOV, top-selling item, busiest hour, weekday/weekend split and break-even all derivable from the order log (Part B metrics)
O-8	(Stage 3) Natural-language ordering	Agent extracts all 7 fields, confirms each, handles ambiguity, and falls back gracefully
O-9	(Stage 3) Digital adoption	Share of orders placed through the system (vs phone) trends up after launch

Primary outcome: SliceMatic moves from no order data and manual, error-prone billing to consistent automated billing with a complete, analysable record of every order — the foundation for every downstream business decision.

9. Assumptions

1. One order = one Base + one Pizza + one Topping, multiplied by quantity (per the client's reference bill).
2. The 10% discount applies to the pre-tax subtotal when quantity ≥ 5 ; GST is then charged on the post-discount amount.
3. GST is a flat 18% on home delivery, added at billing, excluded from the P&L (pass-through).
4. Prices in menu files are GST-exclusive.
5. Currency is INR throughout; values rounded to 2 decimals.

10. Open Questions — Clarification Needed from Client

These are genuine ambiguities in the brief that affect the build. We recommend resolving them before development starts; defaults we will assume (if no answer) are noted.

#	Question	Why it matters	Default if unanswered
Q1	Should we capture a delivery address (and is delivery in scope for this version)?	A delivery business can't deliver without one; impacts data model and flow.	Out of scope for MVP; name + phone only.
Q2	Can one order contain multiple different pizzas (a true cart), or is it one configuration × quantity?	Changes the entire order/data model and bill structure.	One configuration × quantity (matches reference bill).
Q3	Is a topping mandatory , and can a customer pick more than one topping (or none)?	Reference model says "avg 1 topping"; the brief lists "Topping" singular. Affects pricing and selection step.	Exactly one mandatory topping per pizza.
Q4	For Card/UPI , do we need real payment processing now, or is selection + confirmation sufficient for this version?	Real gateways need integration, keys, reconciliation.	Selection + confirmation only (no live payment) for MVP.
Q5	Should the discount be configurable (threshold/percentage) or hard-fixed at qty ≥ 5 / 10%?	Owner may want to run promotions; graders also test changing the threshold.	Keep the rule in one place so it can be changed easily; default qty ≥ 5 / 10%.
Q6	What should happen to a partially completed order the customer abandons before payment?	Affects whether/how incomplete sessions are logged.	Not written to orders_log.txt; only completed orders are persisted.

11. Out of Scope (this version)

1. Aggregator (Zomato/Swiggy) integration
2. Loyalty/coupon codes beyond the qty discount
3. Multi-outlet support
4. Live order tracking
5. Inventory management
6. Refunds/cancellations
7. SMS/email notifications
8. Admin dashboard — login, order filters, revenue/sales summaries, CSV export (deferred to Stage 3; see flow 4.2)

SliceMatic — Business Unit Economics

Project: SliceMatic Digital Ordering System

Stage: 1 (Part B of 2) — Business Unit Economics

Prepared for: SliceMatic (New Ashok Nagar, Delhi) — Founder: Mr. Rajan Sharma

Document status: Draft v1.0 for client review

Date: 23 June 2026

0. Approach

This is a financial model for a single-outlet pizza delivery business in East Delhi. We take the client's reference baseline (FY 2024–25 model) as the starting point, reproduce its core unit economics, and then **stress-test it** against the six scenarios the business actually cares about (rent shocks, aggregator commissions, ingredient inflation, etc.). Where we refine a baseline number, the justification is noted inline. All figures are in INR; the P&L is computed

ex-GST (GST is a pass-through — see Section 8).

Headline numbers (baseline, 60% capacity):

Metric	Value
Average Order Value (AOV)	₹847
Monthly revenue (ex-GST)	~₹9.96 L
Contribution margin / order	₹644 (76%)
Total monthly fixed costs	₹2,02,910
Break-even	~315 orders/mo ≈ 11/day
Operating profit (EBITDA)	~₹7.0 L/mo
Payback period	~18 months

1. Monthly Fixed Costs

Costs incurred regardless of order volume — the minimum monthly cash outflow before any profit.

Cost item	Detail	Monthly (₹)
Kitchen rent	1,200 sq ft commercial lease, New Ashok Nagar	55,000
Equipment EMI	2-deck oven, mixer, refrigeration — 36-mo loan on ₹4.5L	14,500
Electricity	Commercial rate, ~1,800 units/mo	12,600
Gas / LPG	3 commercial cylinders @ ₹1,850	5,550
Head chef	1 experienced pizza chef	28,000
Kitchen helper	1 prep staff (min wage + ESI/PF)	16,500
Counter + billing	1 person	18,000
Delivery riders	2 riders, fixed component + fuel allowance	32,000
Internet + POS + SaaS	Broadband + POS + gateway annual fee amortised	2,800
Marketing (fixed)	Meta Ads retainer, pamphlets, GBP boost	8,000
Packaging (fixed component)	Branded boxes/bags minimum order	4,200
Misc / contingency (3%)	Maintenance, repairs, consumables	5,760
Total fixed costs		2,02,910

Note: Rider figure is the ****fixed**** salary component only. The per-delivery incentive (₹15/order) is a ****variable**** cost (Section 5).

2. COGS per Pizza — by Pizza Type

COGS per pizza = raw ingredients + packaging + per-order delivery variable. (Wholesale rates, INA Market / Azadpur Mandi.)

Component	Margherita (Thin)	Farm House (Thick)	Cheese Burst (premium)	Avg across menu
Pizza base ingredients	38	45	72	52
Sauce + seasoning	12	14	14	13
Cheese (mozzarella)	28	32	65	38
Pizza-specific toppings	8	22	18	17
Add-on topping (avg 1)	10	10	10	10
Packaging (box + bag)	18	18	18	18
Delivery variable	22	22	22	22
Total COGS / pizza	136	163	219	170

3. Gross Margin — per Pizza and by Category

Pizza	Selling price	COGS	Gross margin	GM %
Margherita (Thin)	397	136	261	65.7%
Farm House (Thick)	417	163	254	60.9%
Cheese Burst (premium)	447	219	228	51.0%
Average	420	170	250	59.5%

\ Selling price = cheapest base + pizza + 1 average topping. Actual order value varies; AOV of ₹847 reflects multi-pizza orders and premium selections.

Margin by category split (illustrative, per average order line):

Category	Avg menu price	Approx. food cost	Approx. GM %
Base	~₹177 (avg of 5)	₹38–72	high (~65–75%)
Pizza	~₹342 (avg of 8)	₹20–55	high (~80%+)
Topping	~₹50 (avg of 10)	~₹10	very high (~80%)

Insight (challenge to baseline): Margin is **inversely driven by cheese intensity**. Cheese Burst's gross margin (51%) is ~15 pts below Margherita's (65.7%) purely because cheese cost jumps ₹28 → ₹65. The menu's profit lever is therefore **mix**, not price: nudging customers toward thin/thick bases and high-margin toppings lifts blended margin more than a price rise would. This is a direct input to the Stage 3 recommendation engine.

4. Revenue Model — Monthly Projections (60% capacity)

Metric	Weekday	Weekend/Holiday	Monthly total
Days in period	22	8	30
Orders/day (60% cap)	38	68	~47 avg
AOV	₹792	₹940	₹847
Daily revenue	₹30,096	₹63,920	~₹40,500 avg
Monthly gross revenue	₹6,62,112	₹5,11,360	₹11,73,472
GST collected (18%, → Govt)			₹1,77,564
Net revenue (ex-GST)			₹9,95,908

Max capacity = 80 orders/day (1 pizza every 6 min, 2-oven setup). 60% utilisation is a deliberately conservative launch assumption. At 100% capacity revenue $\approx$ ₹19.6 L/mo.

5. Contribution Margin & Break-Even

Contribution margin per order = revenue (ex-GST) – **all** variable costs per order. It measures how much each order contributes toward fixed costs and then profit.

P&L line	Per order (₹)	Monthly @ 60% (₹)
Revenue (ex-GST)	847	11,88,996
– Ingredient COGS	(148)	(2,07,768)
– Packaging (variable)	(18)	(25,272)
– Delivery variable (rider incentive)	(22)	(30,888)
– Payment gateway fee (1.8% of GMV)	(15)	(21,402)
Contribution margin	644 (76%)	9,03,666
– Total fixed costs		(2,02,910)
Operating profit (EBITDA)		7,00,756
Operating margin		58.9%

Break-even:

Fixed costs/mo	CM/order	Break-even/mo	Break-even/day
₹2,02,910	₹644	315 orders	~11/day

At 60% capacity (47/day) the outlet runs at **4.3× break-even** — a wide safety margin. It only turns unprofitable below ~11 orders/day (a 76% collapse from plan).

Refinement / caveat (critical note): the baseline counts ingredient COGS at ₹148/order while the per-pizza table shows ~₹130 food/pizza, and AOV ₹847 implies **>1 pizza per order**. So the "per order" variable line is conservative for a single pizza but **understates** food cost for genuinely multi-pizza orders (food scales with pizzas; delivery and gateway scale with orders). For order-level accuracy we recommend the model scale ingredient + packaging by pizza count and keep delivery as a per-order cost. We retain the baseline figures here for comparability and flag this for refinement.

6. GST Treatment — How 18% Flows Through the P&L

Home delivery of restaurant food attracts **18% GST** (no composition scheme). SliceMatic is GST-registered.

- **Who absorbs it? The customer.** GST is added on top of the bill; SliceMatic collects it on the government's behalf. It is **never the shop's money**.
- **It does not flow through profit.** GST collected is a liability, not revenue. The P&L is always computed on the **ex-GST** amount.
- **Input Tax Credit (ITC):** SliceMatic reclaims GST paid on ingredients, packaging and equipment (~₹18,000–22,000/mo), netted against GST collected.
- **Effective GST liability:** Output GST (~₹1,77,564/mo) – ITC (~₹20,000/mo) $\approx$ **₹1,57,564/mo** net remittance.
- **Pricing implication for the app:** menu prices are GST-**exclusive**; GST is added once, at the billing stage, on the **post-discount** total (not per item).

Sample bill (Stage 2 reference):

Line	₹
Base: Cheese Burst	229.00
Pizza: BBQ Chicken	379.00
Topping: Extra Cheese	69.00
Unit subtotal	677.00
× Qty 5 → Subtotal	3,385.00
Discount 10% (qty ≥ 5)	(338.50)
Post-discount subtotal	3,046.50
GST @ 18%	548.37
Total payable	3,594.87

7. Stress Tests — "Challenge These Numbers"

The six scenarios the model must answer. Each shows the method so the result can be reproduced.

Q1 — Rent rises to ₹70,000/mo. New break-even? At what rent is the model unviable?

- New fixed costs = 2,02,910 – 55,000 + 70,000 = **₹2,17,910**.
- New break-even = 2,17,910 / 644 = **338 orders/mo ≈ 11.3/day → ~12/day**.
- **Break-even barely moves (11 → ~12/day)**. A ₹15k rent hike costs only ~1 extra order/day.

At what rent does it become unviable? Define "unviable" as break-even reaching the **planned operating volume** (47/day ≈ 1,404 orders/mo). That requires fixed costs to absorb the full contribution at plan: 1,404 × 644 ≈ ₹9.04 L. Non-rent fixed costs are ₹1,47,910, so rent would have to reach **~₹7.6 L/month** — about **13× current rent** — before the business stops clearing its costs at planned volume.

Conclusion: thanks to the 76% contribution margin, the model is **extremely robust to rent**. Rent is not the binding risk — **demand volume is**.

Q2 — 40% of orders via aggregator at 25% commission. New CM/order and break-even?

Assumptions: aggregator takes 25% of the ex-GST order value and handles **delivery and payment** (so on those orders SliceMatic saves the ₹22 rider incentive and ₹15 gateway fee). Direct orders unchanged.

- **Aggregator order CM** = 847 – 148 (ingredients) – 18 (packaging) – 211.75 (25% commission) = **₹469.25** (a ~27% hit vs direct).
- **Direct order CM** = ₹644 (unchanged).
- **Blended CM** = 0.4 × 469.25 + 0.6 × 644 = **₹574.10/order**.
- **New break-even** = 2,02,910 / 574.10 = **354 orders/mo ≈ 11.8/day → ~12/day**.

Conclusion: the 25% commission slashes margin **on aggregator orders specifically** (₹644 → ₹469), but because direct orders dominate, blended CM only falls ~11% and break-even rises just ~1/day. Aggregators are a viable **demand channel** — but every order pushed to direct (own app) is worth ~₹175 more in contribution, which is the core economic argument for building this system.

Q3 — At what daily volume is a 3rd rider justified?

- **Financial test:** a 3rd rider adds ~₹16,000/mo fixed. At ₹644 CM/order it pays for itself with 16,000 / 644 ≈ **25 extra orders/month (<1/day)** — financially trivial.
- **Physical test (the real constraint):** 2 riders at 30–35 trips/day ≈ **60–70 deliveries/day** capacity. The 30-min SLA starts to break as volume approaches that ceiling.

Conclusion: hire the 3rd rider when daily orders reach **~55–60/day** — before the 2-rider fleet saturates — to protect the SLA, not because of cost. The trigger is **capacity/SLA, not economics**. (This also unlocks the 5 km

radius.)

Q4 — Impact of the qty ≥ 5 / 10% discount on a 5-pizza order. Value-driver or leak?

Using the sample 5× Cheese Burst order (worst-case margin):

	No discount	With 10% discount
Revenue (ex-GST)	3,385.00	3,046.50
Variable costs (food 5×179 + pkg 5×18 + 1 delivery + gateway)	~1,079	~1,072
Contribution margin	~2,306	~1,975

- The discount removes **~₹331 of contribution** from this order ($\approx$ the ₹338.50 discount, less a tiny gateway saving). CM% drops from ~68% to ~65%.
- **But** compare a 4-pizza order (no discount, CM $\approx$ ₹1,840) to a discounted 5-pizza order (CM $\approx$ ₹1,975): if the discount **converts a 4-pizza intent into 5**, total contribution still **rises by ~₹134**.

Conclusion: it's both — depending on behaviour. It is a **value-driver** when it lifts a customer across the threshold (4→5), and a **margin leak** ($\sim$ ₹331/order) on customers who would have bought 5+ anyway.

Recommendation: keep the discount **configurable**, watch the order-size distribution — heavy clustering exactly at 5 suggests it's driving behaviour (keep it); if 5+ orders occur regardless, trial a smaller % or a free high-margin topping instead (cheaper to give, same perceived value).

Q5 — COGS +12% (ingredient inflation). Extra daily orders to hold EBITDA flat?

- Ingredient COGS 148 $\rightarrow 148 \times 1.12 = 165.76$, i.e. **+₹17.76/order**.
- New CM/order = 644 – 17.76 = **₹626.24**.
- To hold EBITDA at ₹7,00,756: orders $\times 626.24 - 2,02,910 = 700,756 \rightarrow$ orders = 9,03,666 / 626.24 = **1,443/mo $\approx$ 48.1/day** (vs 1,403/mo $\approx$ 47/day baseline).
- **Additional needed = ~40 orders/month $\approx$ 1.3 extra orders/day.**

Conclusion: a 12% ingredient shock is absorbed by **barely more than one extra order per day** — again a function of the high contribution margin. (Applying 12% to all COGS components gives $\sim$ 1.5/day — same order of magnitude.) Ingredient inflation is a **low-severity** risk for this model.

Q6 — Three BI metrics from the order log that directly improve profitability

1. **Basket / AOV analysis (combo + topping attach rate).** Which base+pizza+topping combinations and upsells drive higher AOV. Because fixed costs are flat, every rupee of incremental AOV drops almost straight to profit $\rightarrow$ guides upsell prompts and menu engineering.
2. **Demand by hour $\times$ day-of-week (peak heatmap).** Reveals true peaks so staff and rider scheduling match demand — cuts idle labour cost and prevents SLA-breaching overload. (Feeds the Stage 3 demand-forecasting feature directly.)
3. **Item-level contribution-margin ranking.** Top sellers vs most profitable items — promote high-margin lines, reprice/retire low-margin ones (e.g. the Cheese Burst margin drag), improving blended margin without touching volume.